Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА

«ТЕСТИРОВАНИЕ ПРОГРАММ МЕТОДАМИ «БЕЛОГО ЯЩИКА»»

по дисциплине: «Тестирование и отладка программного обеспечения»

Выполнил: студент группы ИУК4-82Б _____ Губин Е.В.
(Подпись) (И.О. Фамилия)

Проверил: _____ Ерохин И.И.
(Подпись) (И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026

Цель: формирование практических навыков составления набора тестовых данных для структурного тестирования.

Задачи: для заданной процедуры составить граф–схему и тестовые наборы для тестирования маршрутов по определенным критериям.

Вариант №5

Ход выполнения работы

Задача №1

Листинг программы:

```
from collections import defaultdict, deque

class ControlFlowGraph:
    def __init__(self, edges):
        self.edges = edges
        self.nodes = self._get_nodes()
        self.adj = self._build_adj_list()

    def _get_nodes(self):
        nodes = set()
        for u, v in self.edges:
            nodes.add(u)
            nodes.add(v)
        return nodes

    def _build_adj_list(self):
        adj = defaultdict(list)
        for u, v in self.edges:
            adj[u].append(v)
            adj[v].append(u)  # для поиска компонент (неориентированный обход)
        return adj

    def count_components(self):
        visited = set()
        components = 0

        for node in self.nodes:
            if node not in visited:
                components += 1
                queue = deque([node])
                while queue:
                    current = queue.popleft()
                    if current not in visited:
                        visited.add(current)
                        queue.extend(self.adj[current])
        return components

    def cyclomatic_complexity(self):
        E = len(self.edges)
        N = len(self.nodes)
        P = self.count_components()
        V = E - N + 2 * P
        return V, E, N, P

# ========================
# Граф A
edges_A = [
    (1,2),(2,3),(3,4),(4,2),
    (2,5),(5,6),(6,7),(4,7),
    (5,8),(7,9),(8,9)
]
```

```python
# Граф Б
edges_B = [
    (1,2),(2,3),(3,4),
    (2,5),(5,8),
    (3,6),(6,8),
    (4,7),(7,8),
    (7,9),(9,10),
    (8,10)
]

# Граф В
edges_C = [
    (1,2),(2,3),(3,4),
    (2,5),(5,6),(6,7),
    (7,8),(4,8),
    (8,9),(9,11),
    (10,3),(10,9)
]

graphs = {
    "А": edges_A,
    "Б": edges_B,
    "В": edges_C
}

for name, edges in graphs.items():
    cfg = ControlFlowGraph(edges)
    V, E, N, P = cfg.cyclomatic_complexity()
    print(f"Граф {name}:")
    print(f"  Вершины (N) = {N}")
    print(f"  Рёбра (E) = {E}")
    print(f"  Компоненты (P) = {P}")
    print(f"  Цикломатическая сложность V(G) = {V}")
    print()
```

Результаты выполнения:

```
Граф А:
  Вершины (N) = 9
  Рёбра (E) = 11
  Компоненты (P) = 1
  Цикломатическая сложность V(G) = 4

Граф Б:
  Вершины (N) = 10
  Рёбра (E) = 12
  Компоненты (P) = 1
  Цикломатическая сложность V(G) = 4

Граф В:
  Вершины (N) = 11
  Рёбра (E) = 12
  Компоненты (P) = 1
  Цикломатическая сложность V(G) = 3
```

Задача №2

Листинг программы:

```
from collections import defaultdict

class ControlFlowGraph:
    def __init__(self, edges, start, end):
        self.edges = edges
        self.start = start
        self.end = end
        self.graph = self.build_graph()
        self.nodes = self.get_nodes()

    def build_graph(self):
        graph = defaultdict(list)
        for u, v in self.edges:
            graph[u].append(v)
        return graph

    def get_nodes(self):
        nodes = set()
        for u, v in self.edges:
            nodes.add(u)
            nodes.add(v)
        return nodes

    def cyclomatic_complexity(self):
        E = len(self.edges)
        N = len(self.nodes)
        P = 1  # один связный компонент
        return E - N + 2 * P

    # Поиск всех простых путей DFS
    def find_all_paths(self):
        paths = []

        def dfs(node, path):
            if node == self.end:
                paths.append(path[:])
                return
            for neighbor in self.graph[node]:
                if neighbor not in path:
                    dfs(neighbor, path + [neighbor])

        dfs(self.start, [self.start])
        return paths

    # Формирование независимых путей
    def find_independent_paths(self):
        all_paths = self.find_all_paths()
        independent = []
        used_edges = set()

        for path in all_paths:
            path_edges = set(zip(path, path[1:]))
            if not path_edges.issubset(used_edges):
                independent.append(path)
                used_edges.update(path_edges)

        return independent

# ============================
# Граф A
edges_A = [
    (1,2),(2,3),(3,4),(4,2),
    (2,5),(5,6),(6,7),(4,7),
    (5,8),(7,9),(8,9)
]
```

```python
# Граф Б
edges_B = [
    (1,2),(2,3),(3,4),
    (2,5),(5,8),
    (3,6),(6,8),
    (4,7),(7,8),
    (7,9),(9,10),
    (8,10)
]

# Граф В
edges_C = [
    (1,2),(2,3),(3,4),
    (2,5),(5,6),(6,7),
    (7,8),(4,8),
    (8,9),(9,11),
    (10,3),(10,9)
]

graphs = {
    "А": (edges_A, 1, 9),
    "Б": (edges_B, 1, 10),
    "В": (edges_C, 1, 11)
}

for name, (edges, start, end) in graphs.items():
    cfg = ControlFlowGraph(edges, start, end)

    V = cfg.cyclomatic_complexity()
    independent_paths = cfg.find_independent_paths()

    print(f"\nГраф {name}")
    print(f"Цикломатическая сложность V(G) = {V}")
    print(f"Независимые пути ({len(independent_paths)}):")

    for i, path in enumerate(independent_paths, 1):
        print(f"P{i}: {' -> '.join(map(str, path))}")
```

Результаты выполнения:

```
Граф А
Цикломатическая сложность V(G) = 4
Независимые пути (3):
P1: 1 -> 2 -> 3 -> 4 -> 7 -> 9
P2: 1 -> 2 -> 5 -> 6 -> 7 -> 9
P3: 1 -> 2 -> 5 -> 8 -> 9

Граф Б
Цикломатическая сложность V(G) = 4
Независимые пути (4):
P1: 1 -> 2 -> 3 -> 4 -> 7 -> 8 -> 10
P2: 1 -> 2 -> 3 -> 4 -> 7 -> 9 -> 10
P3: 1 -> 2 -> 3 -> 6 -> 8 -> 10
P4: 1 -> 2 -> 5 -> 8 -> 10

Граф В
Цикломатическая сложность V(G) = 3
Независимые пути (2):
P1: 1 -> 2 -> 3 -> 4 -> 8 -> 9 -> 11
P2: 1 -> 2 -> 5 -> 6 -> 7 -> 8 -> 9 -> 11
```

Задача №3

Листинг программы:

```
from itertools import product

# ----------------------------------------
# 1. Модель процедуры
# ----------------------------------------

def m(a, b, x):
    path = []

    # D1
    if (a > 0) and (b < 0):
        path.append("D1_T")
        x = x + 1
    else:
        path.append("D1_F")

    # D2
    if ((a == 2) or (x > 3)) and (b > -10):
        path.append("D2_T")
        x = x - 1
    else:
        path.append("D2_F")

    return x, tuple(path)

# ----------------------------------------
# 2. Цикломатическая сложность
# ----------------------------------------

def cyclomatic_complexity_edges_nodes():
    E = 7
    N = 6
    P = 1
    return E - N + 2 * P

def cyclomatic_complexity_predicates():
    D = 2
    return D + 1

def cyclomatic_complexity_regions():
    return 3

# ----------------------------------------
# 3. Поиск независимых путей
# ----------------------------------------

def find_independent_paths():
    test_values = {
        "a": [-1, 1, 2],
        "b": [-20, -5, 5],
        "x": [0, 4]
    }

    paths = {}

    for a, b, x in product(test_values["a"],
                           test_values["b"],
                           test_values["x"]):
        _, path = m(a, b, x)
        if path not in paths:
            paths[path] = (a, b, x)

    return paths

# ----------------------------------------
# 4. Запуск
```

```
# ----------------------------------------

print("Цикломатическая сложность:")
print("По формуле E-N+2P:", cyclomatic_complexity_edges_nodes())
print("По формуле D+1:", cyclomatic_complexity_predicates())
print("По числу областей:", cyclomatic_complexity_regions())

print("\nНезависимые пути и тесты:")

paths = find_independent_paths()

for i, (path, values) in enumerate(paths.items(), 1):
    print(f"P{i}: {path}")
    print(f"   Тест: a={values[0]}, b={values[1]}, x={values[2]}")
```

Результаты выполнения:

```
Цикломатическая сложность:
По формуле E-N+2P: 3
По формуле D+1: 3
По числу областей: 3

Независимые пути и тесты:
P1: ('D1_F', 'D2_F')
   Тест: a=-1, b=-20, x=0
P2: ('D1_F', 'D2_T')
   Тест: a=-1, b=-5, x=4
P3: ('D1_T', 'D2_F')
   Тест: a=1, b=-20, x=0
P4: ('D1_T', 'D2_T')
   Тест: a=1, b=-5, x=4
```

Потоковый граф:

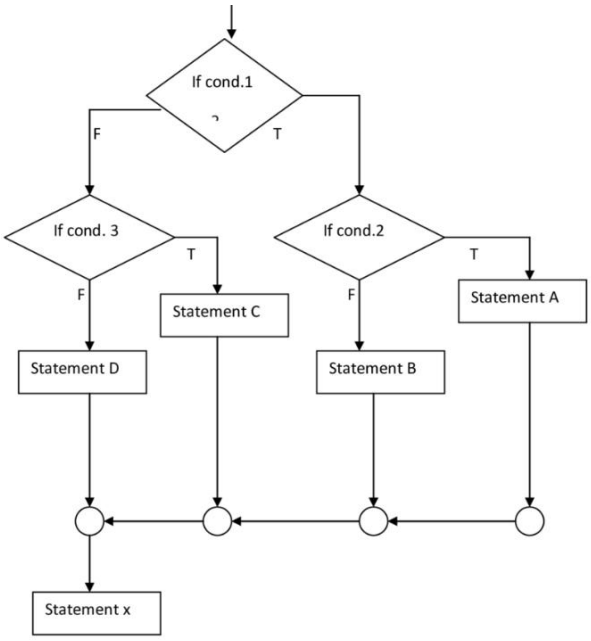
```
Start
  |
 [D1]  a>0 and b<0 ?
  |T          |F
  |           |
 x:=x+1       |
      \      /
        [D2]  ((a=2) or (x>3)) and (b>-10)?
        |T          |F
        |           |
      x:=x-1        |
          \        /
             End
```

Задача №4

Листинг программы:

```
from itertools import product

# ----------------------------------------
# Модель процедуры с трассировкой
# ----------------------------------------

def m(a, b, x):
    trace = {
        "D1": None,
        "D2": None,
        "C1": a > 0,
        "C2": b < 0,
        "C3": a == 2,
        "C4": x > 3,
        "C5": b > -10,
        "S1": False,
        "S2": False
    }

    # D1
    if (trace["C1"]) and (trace["C2"]):
        trace["D1"] = True
        x = x + 1
        trace["S1"] = True
    else:
        trace["D1"] = False

    # D2
    if ((trace["C3"]) or (trace["C4"])) and (trace["C5"]):
        trace["D2"] = True
        x = x - 1
        trace["S2"] = True
    else:
        trace["D2"] = False

    return x, trace
```

Граф-схема процедуры:



Листинг генерации тестового пространства:

```
# Тестовое пространство
values = {
    "a": [-1, 1, 2],
    "b": [-20, -5, 5],
    "x": [0, 4]
}

all_tests = list(product(values["a"], values["b"], values["x"]))
```

Покрытие операторов:

```
def statement_coverage():
    covered_S1 = False
    covered_S2 = False
    tests = []

    for a,b,x in all_tests:
        _, trace = m(a,b,x)

        if trace["S1"] and not covered_S1:
            covered_S1 = True
            tests.append((a,b,x))

        if trace["S2"] and not covered_S2:
            covered_S2 = True
            tests.append((a,b,x))

        if covered_S1 and covered_S2:
            break

    return tests
```

Покрытие решений:

```
def decision_coverage():
    outcomes = set()
    tests = []

    for a,b,x in all_tests:
        _, trace = m(a,b,x)
        key = (trace["D1"], trace["D2"])

        if key not in outcomes:
            outcomes.add(key)
            tests.append((a,b,x))

        if len(outcomes) == 4:
            break

    return tests
```

Покрытие условий:

```
def condition_coverage():
    coverage = {c:set() for c in ["C1","C2","C3","C4","C5"]}
    tests = []

    for a,b,x in all_tests:
        _, trace = m(a,b,x)

        new = False
        for c in coverage:
            if trace[c] not in coverage[c]:
                coverage[c].add(trace[c])
                new = True
```

```
        if new:
            tests.append((a,b,x))

        if all(len(v)==2 for v in coverage.values()):
            break

    return tests
```

Покрытие решений/условий:

```
def decision_condition_coverage():
    tests = set(statement_coverage())
    tests.update(decision_coverage())
    tests.update(condition_coverage())
    return list(tests)
```

Комбинаторное покрытие условий:

```
def combinatorial_coverage():
    combos_D1 = set()
    combos_D2 = set()
    tests = []

    for a,b,x in all_tests:
        _, trace = m(a,b,x)

        c1 = (trace["C1"], trace["C2"])
        c2 = (trace["C3"], trace["C4"], trace["C5"])

        new = False
        if c1 not in combos_D1:
            combos_D1.add(c1)
            new = True

        if c2 not in combos_D2:
            combos_D2.add(c2)
            new = True

        if new:
            tests.append((a,b,x))

        if len(combos_D1)==4 and len(combos_D2)==8:
            break

    return tests
```

Вывод:

Для данной процедуры:
Оптимальный критерий — покрытие решений/условий.
Комбинаторное покрытие избыточно для такой небольшой логики.

Результат выполнения:

```
Statement coverage: [(-1, -5, 4), (1, -20, 0)]
Decision coverage: [(-1, -20, 0), (-1, -5, 4), (1, -20, 0), (1, -5, 4)]
Condition coverage: [(-1, -20, 0), (-1, -20, 4), (-1, -5, 0), (-1, 5, 0), (1, -20, 0), (2, -20, 0)]
Decision/Condition coverage: [(1, -20, 0), (1, -5, 4), (-1, -20, 0), (-1, -5, 4), (-1, 5, 0), (-1, -5, 0), (2, -
20, 0), (-1, -20, 4)]
Combinatorial coverage: [(-1, -20, 0), (-1, -20, 4), (-1, -5, 0), (-1, -5, 4), (-1, 5, 0), (1, -20, 0), (1, 5, 0
), (2, -20, 0), (2, -20, 4), (2, -5, 0), (2, -5, 4)]
```

Задача №5

Листинг программы:

```
from collections import defaultdict

class FlowGraph:
    def __init__(self, edges):
        self.graph = defaultdict(list)
        self.rev_graph = defaultdict(list)
        self.nodes = set()

        for u, v in edges:
            self.graph[u].append(v)
            self.rev_graph[v].append(u)
            self.nodes.add(u)
            self.nodes.add(v)

    # Поиск всех простых циклов (DFS)
    def find_cycles(self):
        visited = set()
        stack = []
        cycles = []

        def dfs(node, start):
            visited.add(node)
            stack.append(node)

            for neighbor in self.graph[node]:
                if neighbor == start:
                    cycles.append(stack.copy())
                elif neighbor not in stack:
                    dfs(neighbor, start)

            stack.pop()

        for node in self.nodes:
            dfs(node, node)

        # Убираем дубликаты циклов
        unique = []
        for c in cycles:
            c_sorted = sorted(c)
            if c_sorted not in unique:
                unique.append(c_sorted)

        return unique

    # Классификация цикла
    def classify_cycle(self, cycle):
        entry_points = 0
        cycle_set = set(cycle)

        for node in cycle:
            for pred in self.rev_graph[node]:
                if pred not in cycle_set:
                    entry_points += 1

        if entry_points == 1:
            return "Естественный цикл (один вход)"
        elif entry_points > 1:
            return "Неструктурированный цикл (несколько входов)"
        else:
            return "Вложенный или внутренний цикл"

    def analyze(self):
        cycles = self.find_cycles()

        if not cycles:
            print("Циклы отсутствуют")
```

```
            return

        for i, cycle in enumerate(cycles, 1):
            print(f"\nЦикл {i}: {cycle}")
            print("Тип:", self.classify_cycle(cycle))

# ============================
# ВЫБЕРИТЕ НУЖНЫЙ ГРАФ
# ============================

edges_5 = [
    (1,2),(2,3),(3,4),(4,5),
    (5,6),(6,7),(7,8),(8,9),(9,10),
    (4,2),
    (8,6),
    (9,5)
]

graph = FlowGraph(edges_5)
graph.analyze()
```

Результаты выполнения:

```
Цикл 1: [2, 3, 4]
Тип: Естественный цикл (один вход)

Цикл 2: [5, 6, 7, 8, 9]
Тип: Естественный цикл (один вход)

Цикл 3: [6, 7, 8]
Тип: Естественный цикл (один вход)
```

Задача №6

Листинг программы:

```
from collections import defaultdict

class FlowGraph:
    def __init__(self, edges):
        self.edges = edges
        self.nodes = set()

        for u, v in edges:
            self.nodes.add(u)
            self.nodes.add(v)

    def cyclomatic_complexity(self, components=1):
        E = len(self.edges)
        N = len(self.nodes)
        P = components

        V = E - N + 2 * P
        return V

# =====================
# Граф из рисунка 5
# =====================

edges_5 = [
    (1,2),(2,3),(3,4),(4,5),
    (5,6),(6,7),(7,8),(8,9),(9,10),
    (4,2),
```

```
    (8,6),
    (9,5)
]

graph = FlowGraph(edges_5)

V = graph.cyclomatic_complexity()

print("Количество вершин:", len(graph.nodes))
print("Количество рёбер:", len(edges_5))
print("Цикломатическая сложность:", V)
print("Необходимое количество наборов тестов:", V)
```

Результаты выполнения:

```
Количество вершин: 10
Количество рёбер: 12
Цикломатическая сложность: 4
Необходимое количество наборов тестов: 4
```

Вывод: в ходе выполнения лабораторной работы были получены практические навыки по написанию тестов по методам «БЕЛОГО ЯЩИКА».